

In my case, to generate items for the invoice, I picked up the latest commits since the last invoice was raised. This produced a draft to edit and was a lot faster than typing all the information manually. 'git log --oneline origin/{target}..{sha}' can be used to show commits since the sha commit. This is a very useful command, indeed!

With a draft ready for the items that have to be put in the invoice, many invoices can be produced quickly. Proofreading and entering the amounts that the client has to pay is all that's left.

Filing the template

Since docx is zip readable and the invoice items are available to use in Python as a data structure, the next step is to combine the two. When we open the docx file using the zipfile library, we need to look for the 'word/document.xml' file within the zip. This is

what we need to edit. The rest can be written back to the new file exactly the same as before. For the 'word/document.xml' file we have to replace the placeholders that we put in the template with actual values.

I initially started with trying Jinja templates (<https://jinja.palletsprojects.com/en/3.0.x/templates/>). That felt like overkill though, so I tried the re library (<https://docs.python.org/3/library/re.html>) in Python. It still felt like overkill.

So the final code I used is 'string.replace('a', 'b')' and it worked well enough for this use case. I just replaced placeholders in the template with actual values. For example, string.replace('BILL_TO', 'ABC Pvt Ltd').

This final replaced docx is written back to the file system. The new docx has been created like a charm.

Making a PDF

A PDF is still needed. Docx can look very different if it is opened in Windows, LibreOffice, or Google Drive. A PDF, though, always looks exactly the same. I used LibreOffice to convert the docx to a PDF.

```
libreoffice --headless \
--convert-to pdf {fname}.docx \
--outdir invoices
```

This PDF can be created in roughly 5 seconds, reducing the time spent in creating invoices from 1-1.5 hours to just 5 minutes of proofreading.

All the code for this tutorial is available at (a gist on GitHub) <https://gist.github.com/theSage21/29261e071efd92f701b69ddd56374fa6>. **END** 🐧

 **By: Arjoonn Sharma**

The author is an open source enthusiast, and likes working with Python and Rust.

Continued from page...73

If you try to print the secret value without the 'AMBER_SECRET' environment variable, it will result in an error, as expected:

```
> set -e AMBER_SECRET
> amber print
Error: Error loading secret key from environment variable
AMBER_SECRET
```

Caused by:
environment variable not found

This is because you need 'AMBER_SECRET' to decrypt the value.

There are various other sub-commands provided by Amber, which can be looked up and experimented with using the '--help' command. The 'exec' is quite handy to propagate secrets as environment variables. (Make sure to export the 'AMBER_SECRET' environment variable before you run the command). Using the 'exec'

sub-command, we can print the secret variable:

```
> amber exec printenv SECRET_VAR1
*****
```

You can see that while printing the secret value, the code masks it automatically.

You can experiment with other sub-commands too, to explore Amber's functionality. Amber can be really helpful when you need simple solutions for managing secrets. **END** 🐧

Reference

<https://www.fpcomplete.com/blog/announcing-amber-ci-secret-tool/>

 **By: Sibi Prabakaran**

The author is a senior engineer working at FPComplete.